

File Upload API

Project Documentation

Prepared by:

Prashant Gautam

Nihit Verma

Archisman Basu

Date: October 2025

1. Project Overview

The File Upload API is a Spring Boot-based backend system designed to securely upload, store, and manage CSV files. It integrates encryption and metadata management features to ensure data confidentiality and integrity. The system supports uploading CSV files, storing them in an encrypted format, retrieving decrypted previews, and enabling secure downloads.

2. Objectives

- Create a secure file handling system for CSV uploads.
- Implement AES encryption for sensitive data storage.
- Ensure smooth retrieval, decryption, and download of stored files.
- Maintain metadata such as filename, upload date, checksum, and size.

3. System Architecture

The system follows a modular architecture with clearly separated layers:

- Controller Layer: Handles REST API requests and responses.
- Service Layer: Manages file encryption, storage, and retrieval.
- Repository Layer: Persists metadata in MongoDB.
- Utility Layer: Provides cryptographic utilities for AES encryption/decryption and checksum.

Files are uploaded as multipart/form-data, encrypted using AES with a secret key and IV, and stored in a secure directory. Metadata is recorded in MongoDB for tracking.

4. API Endpoints

POST /api/files/upload

Description: Uploads a CSV file and encrypts it.

Parameters: file (Multipart), template (String: payments/invoice/transactions)

Response: JSON metadata with filename, checksum, and timestamp.

GET /api/files/view/{template}

Description: Returns first 20 lines of the decrypted CSV file as text.

Parameters: template (String)

Response: JSON array of lines.

GET /api/files/download/{template}

Description: Downloads the decrypted CSV file for the provided template.

Parameters: template (String)

Response: CSV file as attachment.

5. Technology Stack

- Backend Framework: Spring Boot 3.1.4
- Programming Language: Java 21
- Database: MongoDB
- Build Tool: Maven
- Encryption: AES (Advanced Encryption Standard)
- Testing: JUnit, Mockito
- Tools: IntelliJ IDEA, Postman

6. Project Structure

file-upload-api/

```
|-- src/
|   |-- main/java/com/bank/uploadapi/
|   |   |-- controller/FileController.java
|   |   |-- service/FileService.java
|   |   |-- entity/FileMetadata.java
|   |   |-- util/CryptoUtil.java
|   |-- resources/application.properties
|-- uploads/ (encrypted files stored here)
|-- pom.xml
|-- README.md
```

7. Configuration & Running

1. Clone the project and open it in IntelliJ IDEA.
2. Configure MongoDB connection in application.properties.
3. Set AES key and IV for encryption.
4. Run using: `mvn spring-boot:run`
5. Access API endpoints on `http://localhost:8080`.
6. Use Postman or a frontend form for file uploads.

8. Security Measures

- AES encryption ensures data confidentiality.
- Unique IV and key configuration per deployment.
- Secure upload directory with restricted access.
- Validation checks for file types and templates.
- Metadata stored safely in MongoDB.

9. Conclusion

The File Upload API provides a secure and efficient way to handle file uploads in enterprise environments. With encryption, metadata management, and modular design, the system achieves high reliability and integrity for handling sensitive data.